

VM Console — CI/CD Command Reference

All commands actually used to set up, deploy, and troubleshoot this pipeline, organized by phase.

This is a reference, not a script to run top-to-bottom — some commands are one-time setup, some are repeated troubleshooting steps, and several need real values (IPs, keys, passwords) substituted in.

Run WSL-side commands from your WSL terminal (`suraj_patil@DESKTOP-300H0H3`).

Run EC2-side commands only after SSHing into the relevant instance — check your prompt before running anything; this project hit repeated bugs from running EC2 commands on WSL or vice versa.

1. Self-Hosted Runner Setup (WSL, one-time)

Code (bash)

```
# Create a clean, native-WSL location (NOT /mnt/c/... — path spaces break native
module builds
# and /mnt/c/ access is slower than native WSL filesystem)
mkdir -p ~/actions-runner && cd ~/actions-runner

# Download the runner (get the exact URL + version from GitHub's own
# Settings -> Actions -> Runners -> New self-hosted runner page — it includes a
checksum)
curl -o actions-runner-linux-x64-2.336.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.336.0/actions-runner-linux-
x64-2.336.0.tar.gz

# Verify checksum (value comes from the same GitHub setup page)
echo "PASTE_CHECKSUM_HERE actions-runner-linux-x64-2.336.0.tar.gz" | shasum -a 256 -c

tar xzf ./actions-runner-linux-x64-2.336.0.tar.gz

# Register — for an ORG-level runner (serves every repo in the org), use the org URL,
not a repo URL:
./config.sh --url https://github.com/Adii-Vulnerability-Management --token
PASTE_TOKEN_FROM_GITHUB

# Install and start as a persistent systemd service — do NOT just run ./run.sh in a
terminal,
# it dies the moment you close that terminal
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

Removing/re-registering a runner (e.g. switching from repo-level to org-level)

Code (bash)

```
cd ~/actions-runner
sudo ./svc.sh stop
sudo ./svc.sh uninstall
./config.sh remove --token PASTE_REMOVAL_TOKEN_FROM_GITHUB
```

2. Docker Desktop / WSL Credential Fix (WSL, one-time, if you hit `docker login` failures)

Code (bash)

```
# Symptom: "UtilAcceptVsock ... accept4 failed" during docker/login-action in CI –
# happens because the systemd-run runner can't reach Docker Desktop's Windows-side
# credential helper the way an interactive terminal can.
cp ~/.docker/config.json ~/.docker/config.json.bak
nano ~/.docker/config.json
# Delete the "credsStore": "desktop.exe" line entirely, leaving just the "auths"
# block.
cat ~/.docker/config.json # verify the change actually saved
```

3. EC2 Instance Setup (run once per instance, right after SSHing in with the `.pem`)

Code (bash)

```
# Fix a downloaded .pem's permissions before first use (WSL/Windows downloads are
# often too open)
mkdir -p ~/.ssh
cp "/mnt/c/Users/Suraj Patil/Downloads/your-key.pem" ~/.ssh/
chmod 400 ~/.ssh/your-key.pem
ssh -i ~/.ssh/your-key.pem ubuntu@<EC2_PUBLIC_IP>

# Install Docker (skip if the AMI already ships it – check with `docker --version`
# first)
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu
# Log out and back in for the group change to apply:
exit
ssh -i ~/.ssh/your-key.pem ubuntu@<EC2_PUBLIC_IP>
docker ps # should run with no sudo, no error

# Confirm the docker.sock GID (needed for the backend's DOCKER_HOST_GID build arg)
getent group docker
```

4. Dedicated Deploy Key Setup (per service — use a SEPARATE key for each so a leak doesn't cross services)

Code (bash)

```
# On WSL – generate the key. Check the printed comment says your WSL
# username@hostname,
# not the EC2 instance's hostname, or you generated it on the wrong machine.
ssh-keygen -t ed25519 -f ~/.ssh/vm_backend_deploy_key -N ""
cat ~/.ssh/vm_backend_deploy_key.pub
```

```
# On the EC2 instance (via the .pem) – authorize the WSL-generated public key
echo "PASTE_THE_PUBLIC_KEY_HERE" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh

# Back on WSL – test it. Should log in with no .pem and no password prompt.
ssh -i ~/.ssh/vm_backend_deploy_key ubuntu@<EC2_PUBLIC_IP>
```

If you get locked out (authorized_keys emptied/corrupted)

Use **AWS Console -> EC2 -> Instances -> [instance] -> Connect -> EC2 Instance Connect** —

this bypasses SSH key auth entirely via a browser terminal, and works even if `authorized_keys` is genuinely broken.

Code (bash)

```
# From that browser session:
cat ~/.ssh/authorized_keys          # see what's actually there
echo "ssh-ed25519 AAAA...your real key... you@yourhost" >> ~/.ssh/authorized_keys
cat ~/.ssh/authorized_keys          # confirm it landed
```

5. EBS Disk Check / Cleanup (backend instance — the scanner-heavy image needs real disk)

Code (bash)

```
df -h /          # check current usage before and after any cleanup
docker system prune -af    # reclaim space from failed/partial pulls and unused layers
```

*(Resizing the EBS volume itself — AWS Console -> EC2 -> Volumes -> Modify Volume, then

`sudo growpart + sudo resize2fs / sudo xfs_growfs` on the instance — was considered but NOT used in this project; disk pressure was solved instead by shrinking the Docker image.)*

6. Git — Checking State Before Pushing (run from the actual repo directory — verify with `pwd`)

Code (bash)

```
git branch -a          # confirm which branch you're actually on and what remotes exist
git status              # confirm what's staged before committing
git remote -v           # confirm which repo you're in

git add <file>
git commit -m "descriptive message"
git push origin <branch>

# Retrigger a workflow with no code change (e.g. after fixing a runner/env issue)
git commit --allow-empty -m "retrigger ci"
git push origin <branch>
```

7. Deploying / Restarting a Container Manually (bypassing CI, e.g. after an env file change)

Code (bash)

```
# On the target EC2 instance:
docker stop vm-backend && docker rm vm-backend
docker run -d --name vm-backend \
  --restart unless-stopped \
  -p 3102:3000 \
  --env-file /home/ubuntu/vm-backend.env \
  -v /var/run/docker.sock:/var/run/docker.sock \
  adiaaws/vm-backend:latest

# Same pattern for frontend, no docker.sock mount needed:
docker stop vm-frontend && docker rm vm-frontend
docker run -d --name vm-frontend \
  --restart unless-stopped \
  -p 3101:3000 \
  adiaaws/vm-frontend:latest
```

8. Verifying a Deploy Actually Worked (don't trust a green checkmark alone)

Code (bash)

```
# On the EC2 instance:
docker ps                # should show "Up" and eventually "(healthy)"
docker logs <container-name> # full history
docker logs --tail 30 <container-name> # just the newest lines, e.g. right after a login attempt

curl -I http://localhost:<mapped-port>/priv/health # from inside the box

# From your own machine's browser, not SSH:
# http://<EC2_PUBLIC_IP>:<port>/priv/health
```

9. Creating the First Admin User (WSL, run from the backend repo directory)

Code (bash)

```
cd /mnt/c/dev/VM_backend

MONGODB_URL="mongodb+srv://<user>:<password>@<cluster>.mongodb.net/grc-dev?appName=Cluster0" \
SEED_ADMIN_EMAIL="youremail@example.com" \
SEED_ADMIN_PASSWORD="ChooseAStrongPasswordHere" \
node seed-admin.js
```

10. Local Dependency/Build Verification (before trusting CI to catch it)

Code (bash)

```
rm -rf node_modules package-lock.json dist
npm install
npm run build
npm run lint
```

11. Checking a GitHub Release Asset Actually Exists (before pinning a version in a Dockerfile)

Code (bash)

```
curl -fsSL -o /dev/null -w "%{http_code}\n"
"https://github.com/<org>/<repo>/releases/download/v<version>/<asset-filename>"
# 200 = exists. 404 = pruned/wrong URL – check the repo's Releases page for the real
current version/org.
```

Notes on secrets used above

Every placeholder like `<EC2_PUBLIC_IP>`, `<password>`, `PASTE_TOKEN_HERE` needs a real value substituted in from your own setup — none of the values used during this project's actual build are repeated here, since several were posted in chat and should be treated as already-exposed (rotate the MongoDB Atlas password if you haven't already).